

ADVANCED PROGRAMMING

ASSIGNMENT 8

NAME- BHITALI BORUAH

ROLLNO-CSB24045

1. Setting Up the Component

- Created a main React component called `EnrollmentDashboard`
- Set up the basic HTML structure with a dashboard layout
- Divided the UI into left panel (student list) and right panel (forms and stats)

2. State Management Implementation

- Used `useState` hook to manage all data
- Created a Map structure to store students with ID as key
- Created state variables for form inputs (name, GPA, course selection)
- Created state variable for filter functionality

3. Adding Sample Data

- Initialized the Map with 5 sample students
- Each student has: id, name, enrolledCourses (as Set), and GPA
- Used Set for courses to ensure uniqueness automatically

4. Add Student Feature

- Created a form with name input and GPA dropdown
- Added course selector chips for available courses
- Implemented toggle functionality to select/deselect courses
- On form submit: created new student object with unique ID
- Updated the Map using spread operator (no direct mutation)
- Reset all form fields after successful addition

5. Remove Student Feature

- Added remove button for each student
- On click: cloned the existing Map
- Deleted the student using their ID
- Set the new Map to state

6. Sorting by GPA

- Converted Map values to array
- Used sort method with custom comparator
- Displayed students in descending order (highest GPA first)

7. Finding Unique Courses

- Used reduce method to iterate through all students
- Added each course to a Set (automatically removes duplicates)
- Converted the Set to array for rendering
- Displayed all unique courses as chips

8. Filter by Course

- Added dropdown select with all unique courses
- Implemented filter logic that shows only students enrolled in selected course
- Used filter method with Set.has() for O(1) lookup
- Added clear filter button to reset

9. UI Components Built

- Student list cards showing name, GPA, and enrolled courses
- Course selector chips with visual feedback
- Filter dropdown with all available courses
- Statistics section showing total students, average GPA, and unique courses
- Remove buttons for each student

10. Performance Consideration

- Added note about time complexity ($O(n)$ for filtering)
- Used efficient data structures (Map for lookup, Set for uniqueness)

11. Key Programming Concepts Used

- map: To convert student Map to array for rendering
- filter: To filter students by selected course
- reduce: To collect all unique courses
- spread operator: For immutable state updates
- Set to Array conversion: Before rendering in JSX
- No direct state mutation: Always created new objects/arrays

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Course Enrollment Dashboard</title>
  <script crossorigin
src="https://unpkg.com/react@18.2.0/umd/react.development.js"></script>
  <script crossorigin
src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.development.js"></sc
ript>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
  <style>
    * { box-sizing: border-box; font-family: system-ui, -apple-system,
'Segoe UI', Roboto, sans-serif; }
    body { background: #f4f6fa; display: flex; justify-content: center;
padding: 1.5rem; margin: 0; }
    .dashboard { max-width: 1100px; width: 100%; background: white;
border-radius: 28px; box-shadow: 0 12px 30px rgba(0,0,0,0.1); padding:
2rem 2rem 2.5rem; }
    h1 { margin-top: 0.2rem; font-weight: 600; font-size: 1.9rem; color:
#1e293b; display: flex; align-items: center; gap: 8px; }
    h1 span { background: #dbeafe; color: #1e4fd9; font-size: 1rem;
padding: 0.2rem 1rem; border-radius: 40px; font-weight: 500; }
    .grid { display: flex; gap: 2rem; flex-wrap: wrap; }
    .left-panel { flex: 2; min-width: 340px; }
    .right-panel { flex: 1.4; min-width: 280px; }
    .card { background: #f9fafc; border-radius: 24px; padding: 1.5rem;
margin-bottom: 1.8rem; border: 1px solid #e9edf4; }
    .card h3 { margin: 0 0 1.2rem 0; font-weight: 500; color: #0f182a;
display: flex; align-items: center; gap: 0.5rem; }
```



```
.student-row { background: white; border-radius: 16px; padding: 0.8rem 1.2rem; margin-bottom: 0.7rem; box-shadow: 0 2px 6px rgba(0,0,0,0.03); border: 1px solid #eef2f6; display: flex; align-items: center; justify-content: space-between; }

.student-info { display: flex; flex-wrap: wrap; align-items: baseline; gap: 0.8rem 1.5rem; }

.student-name { font-weight: 600; color: #17212d; }

.gpa-badge { background: #e6f0ff; color: #004eac; border-radius: 30px; padding: 0.2rem 0.8rem; font-size: 0.8rem; font-weight: 500; }

.courses-tag { display: flex; flex-wrap: wrap; gap: 0.4rem; }

.course-chip { background: #ecfdf3; color: #0b6b3f; padding: 0.2rem 0.8rem; border-radius: 30px; font-size: 0.75rem; font-weight: 500; }

.remove-btn { background: none; border: 1px solid #ffcdd6; color: #b01c3a; border-radius: 30px; padding: 0.3rem 1rem; font-size: 0.75rem; cursor: pointer; transition: 0.15s; }

.remove-btn:hover { background: #ffe9ed; border-color: #b01c3a; }

.filter-bar { display: flex; gap: 0.8rem; flex-wrap: wrap; align-items: center; margin: 1.2rem 0 1.5rem; }

.filter-select { background: white; border: 1px solid #ccd9e9; border-radius: 40px; padding: 0.5rem 1.2rem; font-size: 0.9rem; outline: none; }

.btn { background: #1e293b; border: none; color: white; padding: 0.5rem 1.5rem; border-radius: 40px; font-weight: 500; cursor: pointer; border: 1px solid #1e293b; transition: 0.15s; }

.btn-outline { background: white; color: #1e293b; border: 1px solid #cbd5e1; }

.btn-outline:hover { background: #f1f5f9; }

.add-form { display: flex; flex-direction: column; gap: 1rem; }

.field-row { display: flex; gap: 1rem; align-items: center; flex-wrap: wrap; }

.field-row input, .field-row select { background: white; border: 1px solid #dde3ed; border-radius: 30px; padding: 0.6rem 1.2rem; font-size: 0.9rem; flex: 1 1 120px; }

.course-selector { display: flex; gap: 0.3rem; flex-wrap: wrap; margin: 0.2rem 0; }

.chip-select { background: #f1f4f9; border-radius: 30px; padding: 0.4rem 1rem; font-size: 0.8rem; border: 1px solid transparent; transition: 0.1s; cursor: pointer; user-select: none; }

.chip-select.selected { background: #1e293b; color: white; border-color: #1e293b; }
```

```

    .stats { background: #ffffff; border-radius: 18px; padding: 1.2rem;
border: 1px solid #eef2f6; }

    .stat-item { display: flex; justify-content: space-between;
margin-bottom: 0.7rem; font-size: 0.95rem; }

    .stat-label { color: #475569; }

    .stat-value { font-weight: 600; color: #0f182a; }

    .unique-courses { display: flex; flex-wrap: wrap; gap: 0.4rem;
margin-top: 1rem; }

    .footnote { font-size: 0.75rem; color: #64748b; margin-top: 2.5rem;
border-top: 1px dashed #d3deed; padding-top: 1.2rem; }
  </style>
</head>
<body>
  <div id="root"></div>

  <script type="text/babel">
    function EnrollmentDashboard() {
      const [studentMap, setStudentMap] = React.useState(() => {
        const map = new Map();
        map.set(101, { id: 101, name: 'Olivia Chen', enrolledCourses: new
Set(['CS101', 'MATH202']), gpa: 3.9 });
        map.set(102, { id: 102, name: 'Ethan Rodriguez', enrolledCourses:
new Set(['CS101', 'PHY101', 'DATA201']), gpa: 3.2 });
        map.set(103, { id: 103, name: 'Sophia Kim', enrolledCourses: new
Set(['MATH202', 'ART105']), gpa: 3.7 });
        map.set(104, { id: 104, name: 'Liam Johnson', enrolledCourses: new
Set(['CS101', 'DATA201']), gpa: 2.8 });
        map.set(105, { id: 105, name: 'Maya Patel', enrolledCourses: new
Set(['PHY101', 'MATH202', 'CS101']), gpa: 3.95 });
        return map;
      });

      const [newName, setNewName] = React.useState('');
      const [newGpa, setNewGpa] = React.useState('3.0');
      const availableCourses = ['CS101', 'MATH202', 'PHY101', 'DATA201',
'ART105', 'ENG203'];
      const [selectedCourses, setSelectedCourses] = React.useState(new
Set());
      const [filterCourse, setFilterCourse] = React.useState('');

```

```

const getStudentsArray = () => Array.from(studentMap.values());

const handleAddStudent = (e) => {
  e.preventDefault();
  if (!newName.trim()) return;
  const newId = Date.now() + Math.floor(Math.random() * 1000);
  const newStudent = {
    id: newId,
    name: newName.trim(),
    enrolledCourses: new Set(selectedCourses),
    gpa: parseFloat(newGpa) || 0.0
  };
  setStudentMap(new Map([...studentMap, [newId, newStudent]]));
  setNewName('');
  setNewGpa('3.0');
  setSelectedCourses(new Set());
};

const handleRemoveStudent = (id) => {
  setStudentMap(prevMap => {
    const newMap = new Map(prevMap);
    newMap.delete(id);
    return newMap;
  });
};

const sortedStudents = getStudentsArray().sort((a, b) => b.gpa - a.gpa);

const allCoursesSet = getStudentsArray().reduce((accSet, student) => {
  student.enrolledCourses.forEach(course => accSet.add(course));
  return accSet;
}, new Set());

const uniqueCoursesArray = Array.from(allCoursesSet).sort();

const filteredStudents = sortedStudents.filter(student =>
  filterCourse === '' || student.enrolledCourses.has(filterCourse)
);

```

```

const toggleCourse = (course) => {
  setSelectedCourses(prev => {
    const newSet = new Set(prev);
    newSet.has(course) ? newSet.delete(course) : newSet.add(course);
    return newSet;
  });
};

return (
  <div className="dashboard">
    <h1>📋 course enrollment <span>react · map+set</span></h1>
    <div className="grid">
      <div className="left-panel">
        <div className="card">
          <h3>📌 enrolled students (sorted by GPA ▼)</h3>
          <div className="filter-bar">
            <select
              className="filter-select"
              value={filterCourse}
              onChange={(e) => setFilterCourse(e.target.value)}
            >
              <option value="">🎓 all courses (no filter)</option>
              {uniqueCoursesArray.map(course => (
                <option key={course}
value={course}>{course}</option>
              ))}
            </select>
            {filterCourse && (
              <button className="btn-outline btn" onClick={() =>
setFilterCourse('')}>
                clear filter
              </button>
            )}
          </div>

          {filteredStudents.length === 0 ? (
            <div style={{ color: '#64748b', padding: '1rem 0' }}>🌟
no students match this course</div>
          ) : (

```



```

        filteredStudents.map(student => (
          <div key={student.id} className="student-row">
            <div className="student-info">
              <span
className="student-name">{student.name}</span>
              <span className="gpa-badge">GPA
{student.gpa.toFixed(2)}</span>
              <div className="courses-tag">
                {Array.from(student.enrolledCourses).map(course
=> (
                  <span key={course}
className="course-chip">{course}</span>
                ))}
              </div>
              <button className="remove-btn" onClick={() =>
handleRemoveStudent(student.id)}>X remove</button>
            </div>
          ))
        )}
      </div>
</div>

<div className="right-panel">
  <div className="card">
    <h3>+ add new student</h3>
    <form className="add-form" onSubmit={handleAddStudent}>
      <div className="field-row">
        <input
          type="text"
          placeholder="full name"
          value={newName}
          onChange={(e) => setNewName(e.target.value)}
          required
        />
        <select value={newGpa} onChange={(e) =>
setNewGpa(e.target.value)}>
          <option value="4.0">4.0 (A)</option>
          <option value="3.7">3.7 (A-)</option>
          <option value="3.3">3.3 (B+)</option>

```

```

        <option value="3.0">3.0 (B)</option>
        <option value="2.7">2.7 (B-)</option>
        <option value="2.3">2.3 (C+)</option>
        <option value="2.0">2.0 (C)</option>
    </select>
</div>
<div>
    <div style={{ marginBottom: '0.6rem', fontSize:
'0.9rem', color: '#334155' }}><img alt="📖 icon" data-bbox="428 255 448 268"/> select courses:</div>
    <div className="course-selector">
        {availableCourses.map(course => (
            <span
                key={course}
                className={`chip-select
${selectedCourses.has(course) ? 'selected' : ''}`}
                onClick={() => toggleCourse(course)}
            >
                {course}
            </span>
        ))}
    </div>
</div>
    <button type="submit" className="btn" style={{
marginTop: '0.8rem' }}><img alt="✅ icon" data-bbox="355 560 375 573"/> add student</button>
</form>
</div>

<div className="stats">
    <h3 style={{ marginTop: 0, marginBottom: '1rem' }}><img alt="📊 icon" data-bbox="812 662 832 675"/>
summary</h3>
    <div className="stat-item">
        <span className="stat-label">total students</span>
        <span
className="stat-value">{getStudentsArray().length}</span>
    </div>
    <div className="stat-item">
        <span className="stat-label">average GPA</span>
        <span className="stat-value">
            {(getStudentsArray().reduce((sum, s) => sum + s.gpa,
0) / (getStudentsArray().length || 1)).toFixed(2)}

```

```

        </span>
      </div>
      <div className="stat-item">
        <span className="stat-label">unique courses</span>
        <span
className="stat-value">{uniqueCoursesArray.length}</span>
      </div>
      <div className="unique-courses">
        {uniqueCoursesArray.map(course => (
          <span key={course} className="course-chip" style={{
background: '#e9eef4', color: '#1e293b' }}>{course}</span>
        ))}
      </div>
    </div>

    <div className="footnote">
      ⌚ filtering by course: O(n) · uses Map, Set, reduce,
filter · no mutations
    </div>
  </div>
</div>
</div>
);
}

ReactDOM.render(<EnrollmentDashboard />,
document.getElementById('root'));
</script>
</body>
</html>

```

Output

course enrollment react · map+set

📌 enrolled students (sorted by GPA ▼)

🏠 all courses (no filter) ▼

Rohit GPA 4.00 ENG203 PHY101 DATA201 ✕ remove

Vikas kumar GPA 3.00 ✕ remove

manish GPA 3.00 CS101 MATH202 PHY101 ✕ remove

Rupam kumari GPA 3.00
CS101 MATH202 ENG203 ✕ remove

+ add new student

full name

3.0 (B) ▼

📌 select courses:

CS101

MATH202

PHY101

DATA201

ART105

ENG203

✓ add student

📊 summary

total students 4

average GPA 3.25

unique courses 5

CS101

DATA201

ENG203

MATH202

PHY101

⌚ filtering by course: O(n) - uses Map, Set, reduce, filter - no mutations